SolvencyInsight

AI-Powered Insolvency Prevention & Workforce Optimization Platform

---

Comprehensive Project Report

Final Project Evaluation Documentation

Project Type	Full-Stack AI/ML Web Application
Tech Stack	React 19 + TypeScript | FastAPI + Python | XGBoost + SHAP
Deployment	Docker Compose (Backend + Frontend + Nginx)
Date	April 04, 2026

This document provides an exhaustive technical reference covering architecture, machine learning models, API design, frontend implementation, data engineering, deployment infrastructure, and anticipated evaluator questions.

Table of Contents

1. Executive Summary

SolvencyInsight is a production-grade, full-stack AI platform that predicts company insolvency risk and optimizes workforce decisions. It combines classical financial theory (Altman Z-Score) with modern machine learning (XGBoost with SHAP explanations), market intelligence (news sentiment, sector trends, economic indicators), and interactive visualization.

1.1 Core Capabilities

- **Insolvency Risk Prediction** - XGBoost classifier on 12 financial ratios with Platt-calibrated probabilities and SHAP explainability
- **Altman Z-Score Analysis** - Classical 1968 formula with Safe/Grey/Distress zone classification
- **Employee Attrition Prediction** - XGBoost on 18 features with retention scoring and layoff simulation
- **Enhanced Market-Aware Prediction** - Blends ML base probability with live news sentiment, sector data, and economic indicators
- **PDF Report Generation** - Professional insolvency and layoff simulation reports with ReportLab
- **Company Comparison** - Side-by-side risk analysis of two companies with SHAP waterfall charts
- **Bulk CSV Analysis** - Upload hundreds of companies/employees for batch scoring

1.2 Key Metrics

Metric	Insolvency Model	Employee Model
Accuracy	91.0%	92.0%
Precision	86.4%	70.0%
Recall	76.0%	87.5%
F1 Score	80.9%	77.8%
ROC-AUC	94.1%	97.2%
5-Fold CV AUC	95.8% (+/- 2.4%)	N/A

Model performance on held-out 20% test set (stratified split, seed=42).

2. System Architecture & Technology Stack

2.1 Architecture Overview

The system follows a **3-tier client-server architecture** containerized with Docker Compose:

- **Presentation Tier** - React 19 SPA with TypeScript, Tailwind CSS, Framer Motion animations, Recharts visualization, served by Nginx
- **Application Tier** - FastAPI (Python) with async endpoints, thread-pool execution for ML inference, response caching (TTL=600s, max=2000)
- **ML/Data Tier** - XGBoost models with SHAP explainer, synthetic data generation engine, pickle-serialized model artifacts

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	19.2.0	Component-based UI with hooks
Frontend	TypeScript	5.9.3	Type safety across frontend
Frontend	Vite	7.2.4	Build tool with HMR and code splitting
Frontend	Tailwind CSS	3.4.19	Utility-first styling with custom design system
Frontend	Framer Motion	12.34.0	Page transitions, spring animations, micro-interactions
Frontend	Recharts	3.6.0	Pie, bar, and SHAP waterfall charts
Frontend	Remotion	4.0.409	Programmatic hero video on landing page
Frontend	Axios	1.13.2	HTTP client with interceptors and 60s timeout
Frontend	Lucide React	0.562.0	190+ SVG icon components
Backend	FastAPI	0.104+	Async Python web framework with OpenAPI docs
Backend	Pydantic	2.5+	Request/response validation with Field constraints
Backend	XGBoost	2.0+	Gradient boosted decision trees for classification
Backend	SHAP	0.43+	Shapley Additive Explanations for model interpretability
Backend	scikit-learn	1.3+	CalibratedClassifierCV, cross-validation, metrics
Backend	ReportLab	4.0+	PDF report generation
Backend	httpx + BeautifulSoup	0.25+	Market intelligence API calls and scraping
Infra	Docker Compose	-	Multi-container orchestration
Infra	Nginx	Alpine	Reverse proxy, SPA routing, gzip, static caching
CI/CD	GitHub Actions	-	Lint, test, build, Docker, security scan (Trivy)

2.3 Project File Structure

```
insolvency-prevention-system/
+-- backend/app/main.py # FastAPI app, 24+ endpoints, lifespan
```

```
+-- backend/app/models/schemas.py # 25+ Pydantic models with Field(ge/le) bounds
+-- backend/app/routes/ # financial.py, employee.py, reports.py
+-- backend/app/services/ # market_intelligence, pdf_generator, enhanced_prediction
+-- backend/app/config.py # Pydantic Settings
+-- ml_models/insolvency_predictor.py # XGBoost + Platt + SHAP + validation
+-- ml_models/employee_scorer.py # XGBoost + retention scoring + layoff sim
+-- ml_models/trained_models/ # Serialized .pkl model artifacts
+-- data/generate_dummy_data.py # Synthetic data with accounting identities
+-- frontend/src/pages/ # 7 pages (Landing, Dashboard, Insolvency, ...)
+-- frontend/src/components/ # 18 reusable components
+-- tests/test_api.py # 8 TestClient API tests
+-- tests/test_data_generation.py # 6 data validation tests
+-- docker-compose.yml, Dockerfiles, nginx.conf, .github/workflows/ci.yml
```

3. Machine Learning Models & Algorithms

3.1 Insolvency Prediction Model

3.1.1 Feature Set (12 Financial Ratios)

#	Feature	Formula	Category
1	working_capital_to_total_assets	(CA - CL) / TA	Altman Z (X1)
2	retained_earnings_to_total_assets	RE / TA	Altman Z (X2)
3	ebit_to_total_assets	EBIT / TA	Altman Z (X3)
4	market_value_equity_to_total_liabilities	MVE / TL	Altman Z (X4)
5	sales_to_total_assets	Sales / TA	Altman Z (X5)
6	current_ratio	CA / CL	Liquidity
7	quick_ratio	(CA - Inventory) / CL	Liquidity
8	debt_to_equity	Total Debt / Equity	Leverage
9	interest_coverage	EBIT / Interest Expense	Coverage
10	net_profit_margin	Net Income / Revenue	Profitability
11	return_on_assets	Net Income / TA	Profitability
12	return_on_equity	Net Income / Equity	Profitability

3.1.2 Altman Z-Score Formula (1968)

The classical discriminant function for public manufacturing firms:

```
Z = 1.2*X1 + 1.4*X2 + 3.3*X3 + 0.6*X4 + 1.0*X5
```

Zone	Z-Score Range	Interpretation
Safe	Z > 2.99	Low probability of bankruptcy within 2 years
Grey	1.81 < Z < 2.99	Uncertain zone, warrants monitoring
Distress	Z < 1.81	High probability of financial distress

3.1.3 XGBoost Hyperparameters

Parameter	Value	Rationale
n_estimators	50	Conservative to prevent overfitting on synthetic data
max_depth	3	Shallow trees for smoother decision boundaries
learning_rate	0.05	Low rate + fewer trees = better generalization
reg_alpha (L1)	0.5	Feature selection regularization
reg_lambda (L2)	1.0	Weight shrinkage regularization
min_child_weight	3	Minimum samples per leaf for stability
subsample	0.8	Row sampling to reduce variance

Parameter	Value	Rationale
colsample_bytree	0.8	Feature sampling per tree for diversity
scale_pos_weight	auto	n_negative / n_positive for class imbalance

3.1.4 Feature Preprocessing Pipeline

Before training and at prediction time, the pipeline applies:

- **Missing value imputation** - median imputation per column
- **Feature bounds storage** - 1st and 99th percentile of each feature stored during training
- **Signed log transformation** - sign(x) * log1p(|x|) applied to debt_to_equity and interest_coverage to compress heavy right tails
- **Winsorization** - Input features clipped to [P1, P99] training bounds at prediction time to handle out-of-distribution inputs

3.1.5 Probability Calibration (Platt Scaling)

Raw XGBoost probabilities are often poorly calibrated. We apply **Platt scaling** (sigmoid method) via sklearn's CalibratedClassifierCV with 3-fold cross-validation. This ensures that when the model outputs 30% distress probability, approximately 30% of such companies actually experience distress. This replaces the naive linear mapping (0.02 + 0.96 * p) that was previously used.

3.1.6 Cross-Validated Metrics

In addition to single train/test split metrics, we compute 5-fold stratified cross-validation ROC-AUC to provide robust, unbiased performance estimates. CV AUC mean = 0.958 (+/- 0.024 std).

3.1.7 Model Versioning Metadata

Each trained model artifact stores: training timestamp, number of samples, number of features, bankruptcy ratio, and full hyperparameter dictionary. This answers the question: 'When was this model trained and on what data?'

3.1.8 Input Validation (Accounting Identity Checks)

Before prediction, the system validates input consistency:

- **Quick ratio <= Current ratio** - Quick assets are a subset of current assets
- **DuPont identity** - ROA should approximate NPM x Sales/TA (tolerance: 0.08)
- **WC/CR consistency** - Negative working capital should mean current_ratio < 1
- **D/E vs IC coherence** - Very high leverage with very high interest coverage is unusual

3.1.9 SHAP Explainability

Every prediction includes a SHAP (SHapley Additive exPlanations) breakdown via TreeExplainer. For each input, we provide: raw SHAP values per feature, top 10 risk drivers sorted by |SHAP|, direction of impact (increases/decreases risk), and a base value (expected model output). This makes the model fully interpretable - the examiner can trace exactly why a company is classified as high risk.

3.2 Employee Attrition Model

3.2.1 Feature Set (18 Attributes)

Type	Features
Numeric (13)	age, job_level, performance_rating, job_satisfaction, job_involvement, environment_satisfaction, monthly_income,
Categorical (5)	gender, department, job_role, business_travel, over_time

3.2.2 Retention Score Formula

A domain-driven weighted composite score (0-100):

```
Score = 0.30 * Performance + 0.20 * JobSatisfaction + 0.20 * JobInvolvement +
0.30 * TenureBonus
```

Each component is scaled from its 1-4 rating to 0-100. Tenure bonus is capped at 10 years. Weights are informed by HR analytics literature (Holtom et al., 2008; Griffeth et al., 2000).

3.2.3 Layoff Simulation Algorithm

Given a target budget cut percentage and minimum employees per department:

- Rank employees by layoff priority (High > Medium > Low), then by retention score (ascending)
- Iterate through ranked list, recommending layoff if: (a) cumulative savings < target, and (b) department minimum not reached
- Track cumulative savings, generate reasons (low retention, high priority, high attrition risk)
- Output includes actual savings achieved, department breakdown, and per-employee recommendations

3.3 Enhanced Prediction (Market-Aware)

The enhanced prediction service blends the base ML probability with live market intelligence:

```
adjusted = base + (base * market_adj * 0.5) + (market_adj * 0.3)
```

The first term scales market impact proportionally to existing risk. The second provides a baseline shift. Conservative fixed weights pending calibration against historical data. Market adjustment comes from news sentiment, sector performance, and economic indicators (GDP, unemployment, inflation).

4. Synthetic Data Generation Engine

4.1 Design Philosophy

Since real bankruptcy datasets are small (UCI Polish/Taiwanese ~10K rows), we generate synthetic data with **enforced accounting identities** to ensure financial ratios are internally consistent. This is not random noise - each company has a coherent balance sheet, income statement, and cash flow model.

4.2 Company Data Generation

The **generate_company_data()** function creates companies across 7 industries with these steps:

- **Cohort splitting** - 75% healthy, 25% bankrupt (configurable)
- **Balance sheet construction** - Total assets, current assets, inventory, equity, liabilities generated with industry-calibrated parameters
- **Income statement derivation** - Revenue = asset turnover * TA; EBIT, net income derived from margins
- **Ratio calculation** - All 12 ratios computed FROM the financial statements, not randomly assigned
- **Accounting identity enforcement** - quick_ratio <= current_ratio, ROA = NPM * Sales/TA, WC/CR consistency
- **Z-Score computation** - Altman formula applied to the 5 Z-score components

4.3 Real-World Calibrated Clipping Bounds

Ratio	Min	Max	Non-Financial D/E Max	Source
debt_to_equity	-5	20 (40 for Financial)	20	Damodaran dataset
interest_coverage	-20	50	-	Damodaran dataset
net_profit_margin	-0.8	0.6	-	US public companies
return_on_assets	-0.5	0.4	-	US public companies
return_on_equity	-3.0	3.0	-	US public companies

4.4 Validation Checks (7 Total)

- quick_ratio <= current_ratio (always)
- DuPont identity: ROA ~= NPM x Sales/TA (within 5% tolerance)
- Bankrupt companies have lower Z-scores on average
- Bankrupt companies have higher D/E on average
- Non-financial D/E max <= 20
- EBIT/IC sign consistency (negative EBIT -> negative IC)
- WC/CR consistency (negative WC -> CR < 1)

4.5 Employee Data Generation

The **generate_employee_data()** function uses a causal model:

- Latent **base_dissatisfaction** drives all satisfaction scores (not independent random)

• monthly_income = f(job_level, years_at_company, performance_rating)

• job_level = f(age, total_working_years) - seniority grows with experience

• Temporal chain enforced: years_in_role <= years_at_company <= total_working_years <= (age - 22)

• Attrition probability is a structured weighted sum of causal predictors: overtime (30%), travel (18%), distance (10%), new hire flag (15%), plus noise

5. Backend API & Services

5.1 Complete API Endpoint Reference

Method	Endpoint	Description
GET	/	Root welcome message
GET	/api/health	Health check with model metrics
GET	/api/analyses/recent	Dashboard recent analyses (capped at 50)
GET	/api/templates/company	CSV template download
GET	/api/templates/employee	CSV template download
POST	/api/financial/analyze	Single company analysis with SHAP
POST	/api/financial/upload-single	Single company via CSV
POST	/api/financial/upload	Bulk company analysis
GET	/api/financial/feature-importance	XGBoost feature importances
POST	/api/financial/explain-row	SHAP explanation for one CSV row
POST	/api/employee/analyze	Single employee analysis with SHAP
POST	/api/employee/upload	Bulk employee analysis
POST	/api/employee/simulate-layoff	Layoff simulation with budget constraints
GET	/api/employee/feature-importance	XGBoost feature importances
POST	/api/employee/explain-row	SHAP explanation for one CSV row
POST	/api/reports/generate	PDF report (insolvency or layoff)
POST	/api/reports/insolvency	Insolvency PDF report
POST	/api/reports/layoff	Layoff simulation PDF report
POST	/api/market-intelligence	Market research (news, sector, econ)
POST	/api/financial/analyze-enhanced	ML + market intelligence blend

5.2 Pydantic Schema Validation

All input schemas enforce **real-world bounds** via Pydantic Field(ge=, le=) constraints. For example: current_ratio: ge=0, le=30; debt_to_equity: ge=-10, le=50; interest_coverage: ge=-50, le=100. Employee fields enforce: age 18-70, monthly_income 1000-200000, performance_rating 1-4. Any out-of-range value returns HTTP 422 with specific error details.

5.3 Performance Optimizations

- **Response caching** - SHA256 hash of input data as key, TTL=600s, max=2000 entries, LRU eviction
- **Thread pool execution** - CPU-heavy ML inference runs via asyncio.to_thread() to avoid blocking the event loop
- **File locking** - threading.Lock() prevents race conditions on analysis history JSON

5.4 Market Intelligence Service

Aggregates data from 3 external APIs:

- **NewsAPI** - Industry news with rule-based sentiment analysis (-1 to +1), 15-minute cache
- **FRED** - GDP growth, unemployment, inflation, interest rates, 6-hour cache
- **Alpha Vantage** - Sector performance (1d, 1w, 1m, YTD), hourly cache with simulated fallback

5.5 PDF Report Generation

ReportLab-based service generates two report types:

- **Insolvency Report** - Executive summary, risk metrics, Z-score analysis, SHAP top 10 drivers, recommendations
- **Layoff Report** - Simulation parameters, cost savings analysis, department impact, up to 30 employee recommendations, compliance guidelines

6. Frontend Application

6.1 Pages (7 Total)

Route	Page	Key Features
/	Landing	Remotion hero video, feature cards, CountUp stats, RotatingCube, terminal block
/dashboard	Dashboard	4 stat cards, risk pie chart, dept attrition bar chart, tracked companies, recent activity
/insolvency	Insolvency Analysis	Manual form (13 fields) or CSV upload, RiskGauge, SHAP chart, PDF download, bulk mode
/employees	Employee Scoring	Bulk CSV upload, 6 summary cards, risk pie, dept breakdown, searchable table, detail modal
/layoffs	Layoff Simulation	Budget slider, dept pie, retention comparison, exclusion toggles, CSV export
/reports	Reports	Insolvency + layoff PDF generation, CSV template downloads
/compare	Compare	Side-by-side analysis of 2 companies, dual SHAP charts, comparison bar chart

6.2 Custom Components (18 Total)

Component	Type	Description
Layout	Structural	Sticky header, theme toggle, FloatingNav, background animation
FloatingNav	Navigation	Bottom floating pill with expandable route icons
AnimatedButton	UI Primitive	5 variants (primary/secondary/ghost/danger/success), ripple effect
LoadingSpinner	UI Primitive	Animated spinner with cycling text messages and progress bar
RiskGauge	Visualization	SVG circular gauge (0-100%) with spring animation, risk color coding
ShapChart	Visualization	Horizontal SHAP waterfall - red increases risk, green decreases
CountUp	Animation	Number animation 0 to target, triggers on scroll via useInView
RotatingCube	Animation	3D CSS cube showing Z-Score parameters, 20s rotation
FileUpload	Input	react-dropzone drag-and-drop CSV uploader
ErrorBoundary	Error Handling	React error boundary wrapper

6.3 State Management

Uses React Context API (no Redux):

- **ThemeContext** - Light/dark mode with localStorage persistence
- **ToastContext** - Notification system (success/error/warning/info) with 5s auto-dismiss
- **Local useState** - Each page manages its own form data, loading state, and results
- **useMemo** - Expensive computations (dept breakdowns, filtered tables) are memoized
- **localStorage** - Theme, tracked companies (max 5), first-analysis tutorial flag

6.4 Design System

- **Colors** - Cyan primary (#06b6d4), Blue accent (#3b82f6), Red danger (#ef4444), custom dark palette

• **Typography** - Inter (body), Plus Jakarta Sans (headings), JetBrains Mono (code)

• **Shadows** - primary-glow, highlight-glow, glass morphism effects

• **Dark mode** - Class-based toggle (Tailwind darkMode: 'class')

• **Animations** - Container stagger (0.08s), spring physics (stiffness 200, damping 24), page fade+slide transitions

7. Deployment & DevOps

7.1 Docker Compose Architecture

Service	Image	Port	Health Check
Backend	python:3.11-slim + uvicorn	8000	curl /api/health every 30s
Frontend	node:20-alpine (build) + nginx:alpine	3000 (80 internal)	nginx serves SPA

7.2 Nginx Configuration

- SPA routing: try_files $uri $uri/ /index.html
- Gzip compression enabled for text, JS, CSS, JSON
- Static assets: 1-year cache expiry
- API proxy: /api/ -> backend:8000 with 300s timeout
- Max upload size: 50MB (for large CSV files)

7.3 CI/CD Pipeline (GitHub Actions)

- **Backend job** - Python 3.11: install deps, Ruff linting, pytest
- **Frontend job** - Node 20: npm ci, lint, type-check, build
- **Docker build** - Both images built with layer caching on push to main
- **Security scan** - Trivy vulnerability scanner for CRITICAL and HIGH severity

7.4 Environment Variables

Variable	Purpose	Default
PYTHONDONTWRITEBYTECODE	Prevent .pyc files in containers	1
DATA_SOURCE	synthetic or real (downloads UCI/IBM)	synthetic
SOLVENCY_API_KEY	Optional API key auth for all /api/* routes	empty (disabled)
VITE_API_URL	Frontend API base URL	http://localhost:8000
NEWS_API_KEY	NewsAPI.org for market intelligence	empty
FRED_API_KEY	Federal Reserve economic data	empty
ALPHA_VANTAGE_API_KEY	Sector performance data	empty

8. Testing & Quality Assurance

8.1 Test Suite Summary

Test File	Tests	Framework	What It Tests
tests/test_api.py	8	pytest + TestClient	Root, health, analyze (valid/missing/out-of-range), employee, fe
tests/test_data_generation.py	6	pytest	QR<=CR, DuPont, Z-scores, D/E bounds, class balance, EBIT/

Total: 14 tests, all passing. Tests use FastAPI TestClient (in-process, no server needed) with module-scoped fixture and lifespan context manager for model loading.

8.2 Data Validation (7 Runtime Checks)

The **validate_company_data()** function runs 7 internal consistency checks on generated data and prints [OK] or [X] for each. All 7 pass on the current generation with seed=42.

8.3 Demo Data Files

File	Records	Purpose
test_data/demo_companies.csv	5 companies	Spanning healthy to distressed (Titan Industries -> Omega Corp)
test_data/demo_employees.csv	10 employees	3 depts, 3 high-attrition-risk (low satisfaction, overtime, high distance)

9. Security Hardening

- **Input validation** - Pydantic Field(ge=, le=) on ALL numeric fields prevents garbage-in-garbage-out (e.g., current_ratio=-500 returns HTTP 422)
- **Accounting identity warnings** - Detects impossible inputs (quick_ratio > current_ratio) and warns in the response
- **Optional API key auth** - Set SOLVENCY_API_KEY to require x-api-key header on all /api/* routes (except /health, /docs)
- **File locking** - threading.Lock() prevents race conditions on concurrent analysis history writes
- **Feature winsorization** - Inputs clipped to training distribution [P1, P99] to prevent extrapolation on unseen extremes
- **CORS configured** - Allows all origins in dev; configurable via CORS_ORIGINS env var for production
- **Docker security** - Trivy vulnerability scanning in CI/CD for CRITICAL and HIGH severity issues
- **No secrets in code** - All API keys via environment variables, .env.example documents all options

10. Key Design Decisions & Trade-offs

Synthetic data vs. real datasets

Real bankruptcy datasets (UCI Polish, Taiwanese) are small (~10K) and may not represent all industries. Synthetic data with enforced accounting identities lets us control the data distribution, ensure coverage across 7 industries, and test edge cases. The trade-off is that the model has never seen real-world noise, which is why we use conservative hyperparameters and regularization.

XGBoost vs. neural networks

XGBoost provides: (1) native SHAP support via TreeExplainer (fast, exact), (2) handles tabular data well, (3) interpretable feature importances, (4) works on small datasets. Neural networks would require more data and are less interpretable for financial regulators.

Platt scaling vs. isotonic regression

We chose Platt scaling (sigmoid) because it requires fewer calibration samples and is less prone to overfitting than isotonic regression, which can be noisy with small datasets. Platt scaling assumes a sigmoid relationship between raw scores and true probabilities, which holds well for tree-based models.

Signed log transform vs. standard scaling

debt_to_equity and interest_coverage have heavy right tails. Standard scaling preserves the tail structure, but XGBoost splits on raw values. Signed log (sign(x)*log1p(|x|)) compresses tails while preserving sign, giving the model better split points for extreme values.

React Context vs. Redux

The app has only 2 global states (theme, toasts). Each page is self-contained with local state. Redux would add complexity without benefit. If cross-page state sharing were needed (e.g., a portfolio tracker), Redux or Zustand would be warranted.

Thread pool for ML inference

XGBoost predict and SHAP explain are CPU-bound operations that would block the async event loop. asyncio.to_thread() delegates to a thread pool, keeping the API responsive for concurrent requests.

Route splitting with lazy imports

Routes are split into financial.py, employee.py, reports.py for maintainability. They access main.py globals via sys.modules lookup to handle Python's module path differences between test and production environments.

11. Limitations & Future Scope

11.1 Current Limitations

• **Synthetic training data** - Model has not been validated on real bankruptcy data; performance may differ in production

• **No temporal features** - Model uses point-in-time ratios, not trends (e.g., declining margins over quarters)

• **Single Z-Score formula** - Uses Altman 1968 formula for all industries; Financial Services firms need Z''-Score

• **Employee-company disconnect** - Employee and insolvency models are independent; no company_health_score in employee model

• **Market intelligence requires API keys** - Without keys, enhanced prediction falls back to base model only

• **No database** - Analysis history stored in JSON file (not scalable for production)

• **No user authentication** - Optional API key only; no per-user access control

11.2 Future Enhancements

• Train on real UCI/Taiwanese bankruptcy datasets and validate synthetic-to-real transfer

• Add time-series features (revenue growth rate, margin trend, quarterly EBIT trajectory)

• Implement Z''-Score (1995) for non-manufacturing and service firms

• Add company_health_score as a feature in employee attrition model

• Derived features: DSCR, Cash Flow Adequacy, Operating CF / Total Debt

• PostgreSQL database for scalable analysis history and user management

• JWT-based authentication with role-based access control

• Real-time model monitoring (data drift detection, prediction distribution tracking)

• A/B testing framework for model improvements

12. Expected Evaluator Questions & Model Answers

Q1: Why did you choose FastAPI over Flask or Django?

FastAPI provides: (1) built-in async support for concurrent requests, (2) automatic OpenAPI/Swagger docs, (3) Pydantic integration for request validation with Field(ge=, le=) constraints, (4) native type hints. Flask lacks async; Django is too heavyweight for a pure-API service. FastAPI is also the fastest Python web framework benchmarked (via Starlette/uvicorn).

Q2: Why XGBoost instead of Random Forest, Logistic Regression, or a Neural Network?

XGBoost consistently outperforms Random Forest on tabular data due to gradient boosting's sequential error correction. Logistic Regression cannot capture non-linear interactions between ratios (e.g., high D/E + low IC = crisis). Neural networks require more data and are less interpretable - a critical requirement in finance where regulators demand explainability. XGBoost also has native SHAP TreeExplainer support (exact, fast).

Q3: What is the Altman Z-Score and why is it still relevant?

Edward Altman's 1968 discriminant function (Z = 1.2*X1 + 1.4*X2 + 3.3*X3 + 0.6*X4 + 1.0*X5) was the first successful quantitative bankruptcy predictor. It remains relevant because: (1) it's simple and interpretable, (2) the 5 ratios capture liquidity, profitability, leverage, and efficiency, (3) it's a regulatory standard (Basel Accords reference it), (4) our ML model BUILDS ON it - using the same 5 ratios plus 7 additional features for better discriminative power.

Q4: How do you handle the class imbalance problem? Only ~20-25% of companies go bankrupt.

Three mechanisms: (1) scale_pos_weight = n_negative / n_positive in XGBoost, which increases the loss penalty for misclassifying bankrupt companies, (2) stratified train/test split preserves the class ratio in both sets, (3) we evaluate with ROC-AUC (not accuracy) which is insensitive to class distribution. Accuracy alone would be misleading - predicting 'healthy' for everything gives 75% accuracy.

Q5: What is SHAP and why is it better than feature importance?

SHAP (SHapley Additive exPlanations) is grounded in cooperative game theory. Unlike XGBoost's built-in feature importance (which measures split frequency/gain globally), SHAP provides: (1) per-prediction explanations (why THIS company is risky), (2) direction of impact (increases vs. decreases risk), (3) mathematically guaranteed properties (local accuracy, missingness, consistency). For example, SHAP can show that Company X is high-risk because its D/E=5.2 INCREASES risk by 0.15, while its current_ratio=2.1 DECREASES risk by 0.08.

Q6: Why synthetic data instead of real datasets?

Real bankruptcy datasets (UCI Polish, Taiwanese) have 3 problems: (1) limited size (~10K rows), (2) single-country/industry bias, (3) unknown data quality. Our synthetic generator: (1) produces any volume, (2) covers 7 industries with calibrated parameters, (3) enforces accounting identities (QR<=CR, DuPont, WC/CR), (4) allows controlled experimentation. However, we acknowledge the model needs validation on real data before production deployment. The conservative hyperparameters and regularization mitigate overfitting to synthetic patterns.

Q7: How does the probability calibration work?

Raw XGBoost outputs are often overconfident (clustering near 0 or 1). Platt scaling fits a logistic regression on the model's output scores: P(y=1|f) = 1 / (1 + exp(A*f + B)). We use sklearn's CalibratedClassifierCV with method='sigmoid' and 3-fold CV to learn A and B. The result: when the calibrated model says 30%, approximately 30% of such cases are actually bankrupt. This is essential for risk management - stakeholders need reliable probability estimates, not just rankings.

Q8: What happens if a user enters impossible financial data?

Three layers of defense: (1) Pydantic Field(ge=, le=) rejects values outside real-world bounds immediately (HTTP 422), e.g., current_ratio=-500 is blocked. (2) validate_input() checks accounting identities (quick_ratio > current_ratio is flagged as a warning in the response). (3) Feature winsorization clips inputs to [P1, P99] of the training distribution, so a data entry error like current_ratio=500 is clipped to the 99th percentile before inference.

Q9: How does the system handle concurrent requests?

FastAPI runs on uvicorn (async). ML inference is CPU-bound, so we use asyncio.to_thread() to run predictions in a thread pool, keeping the event loop free for other requests. Analysis history uses threading.Lock() to prevent race conditions on the shared JSON file. The prediction cache (SHA256 key, TTL=600s, max=2000 entries) avoids redundant computation.

Q10: What is the retention score and how is it calculated?

A domain-driven composite: 30% performance (1-4 scaled to 0-100), 20% job satisfaction (1-4), 20% job involvement (1-4), 30% tenure bonus (years at company, capped at 10). The weights come from HR analytics literature (Holtom et al., 2008; Griffeth et al., 2000), which consistently identify performance and tenure as the strongest retention predictors. This is a heuristic, not a fitted model - the XGBoost attrition model provides the ML-based probability.

Q11: How does the market intelligence enhance predictions?

The enhanced prediction blends base ML probability with a market adjustment factor derived from: (1) news sentiment (-1 to +1 from rule-based analysis), (2) sector performance (1d/1w/1m trends), (3) economic indicators (GDP, unemployment, inflation from FRED). The formula is: adjusted = base + base*adj*0.5 + adj*0.3. The first term scales market impact proportionally to existing risk (a risky company is more affected by bad news). The second provides a baseline shift. Coefficients are conservative pending calibration against historical market-to-bankruptcy paired data.

Q12: Why not use a database instead of JSON files for history?

This is a prototype/MVP decision. JSON files work for single-instance deployment and avoid the complexity of database migrations, connection pooling, and ORM setup. The threading.Lock() ensures thread safety for concurrent writes. For production scale, we would migrate to PostgreSQL with SQLAlchemy ORM, which is already in requirements.txt.

Q13: How does the frontend handle large CSV files?

Three mechanisms: (1) Axios timeout set to 60s for file uploads, (2) Backend uses pandas.read_csv on uploaded bytes (efficient for CSVs up to ~50MB, matching Nginx's client_max_body_size), (3) Bulk endpoints return summary statistics + paginated results without SHAP (which is expensive). Users can then request SHAP for individual rows via the explain-row endpoint.

Q14: What are the model's main weaknesses?

Honestly: (1) trained on synthetic data only - needs real-world validation, (2) point-in-time analysis without temporal trends, (3) single Z-Score formula for all industries (financial firms penalized unfairly), (4) no feature interactions explicitly engineered (though XGBoost learns some), (5) enhanced prediction blend weights are heuristic. We document these as limitations and future work.

Q15: How would you deploy this in production?

The system is already containerized. Production deployment would add: (1) PostgreSQL for persistent storage, (2) Redis for distributed caching, (3) JWT authentication with RBAC, (4) Kubernetes or ECS for horizontal scaling, (5) Prometheus + Grafana for monitoring, (6) model registry (MLflow) for versioning, (7) A/B testing for model updates, (8) data drift monitoring on incoming predictions.

Q16: What is the time complexity of a single prediction?

XGBoost predict: O(n_estimators * depth) = O(50 * 3) = O(150) tree traversals per sample. SHAP TreeExplainer: O(T * L * D) where T=trees, L=leaves, D=depth - roughly 10-50ms per sample. Total API latency including validation, preprocessing, prediction, SHAP, and JSON serialization: ~50-200ms for single prediction. Bulk (1000 companies) takes ~2-5 seconds with vectorized operations.

Q17: How do you ensure the model generalizes beyond synthetic data?

Conservative design: (1) shallow trees (depth=3) prevent memorizing patterns, (2) L1+L2 regularization, (3) subsample=0.8 and colsample=0.8 introduce randomness, (4) real-world-calibrated ratio bounds from Damodaran's dataset, (5) accounting identity enforcement ensures financial coherence, (6) 5-fold cross-validation reports robust metrics (CV AUC = 0.958). The next step is validation on real datasets.

Q18: Explain the Docker architecture.

docker-compose.yml defines 2 services on a shared network. Backend: Python 3.11 slim image, runs uvicorn on port 8000, health-checked every 30s via curl /api/health. Frontend: 2-stage build - Node 20 Alpine compiles the Vite app, then copies the dist to nginx:alpine. Nginx serves SPA with try_files for client-side routing, proxies /api/* to backend:8000 with 300s timeout, enables gzip, and sets 1-year cache on static assets.

Q19: What testing strategy have you used?

Two test suites: (1) test_data_generation.py (6 tests) validates synthetic data quality - ratio bounds, accounting identities, class balance, Z-score directionality. (2) test_api.py (8 tests) uses FastAPI TestClient for in-process API testing - valid inputs, missing fields, out-of-range values, employee analysis, feature importance. Both use pytest with module-scoped fixtures. CI/CD runs both suites plus Ruff linting and TypeScript type-checking.

Q20: What would you change if you had more time?

Top 5: (1) Train on real UCI/Taiwanese bankruptcy data and measure synthetic-to-real transfer, (2) Add temporal features (quarterly trends), (3) Implement Z''-Score for service firms, (4) PostgreSQL with SQLAlchemy for proper persistence, (5) Add a model comparison dashboard showing XGBoost vs. Logistic Regression vs. Random Forest on the same data with ROC curves.